

Performance

Anusha N

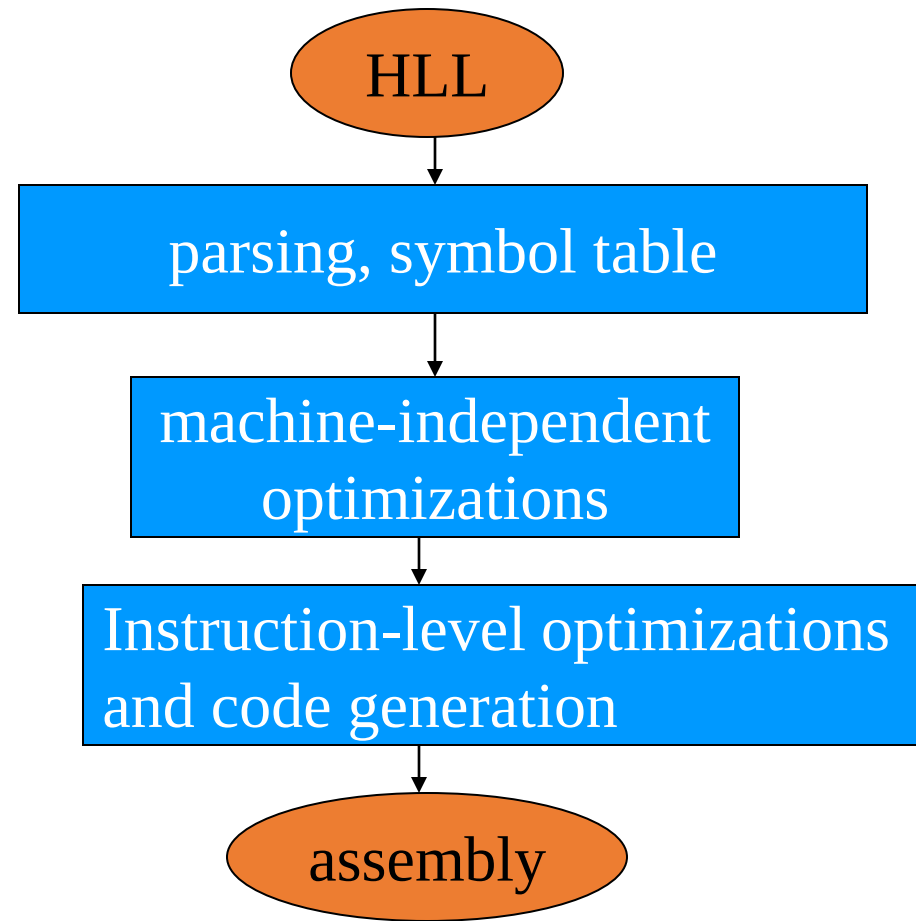
Program design and analysis

- Compilation flow.
- Basic statement translation.
- Basic optimizations.
- Interpreters and just-in-time compilers.

Compilation

- Compilation strategy :
 - compilation = translation + optimization
- Compiler determines quality of code:
 - use of CPU resources;
 - memory access scheduling;
 - code size.

Basic compilation phases



Statement translation and optimization

PARSING

- **Lexical analysis**— Variable and symbol tokens are created
It removes space, comments ,quotations
- **Syntax analysis**— DFG/CDFG created
- **Semantic analysis**—Assembly code generated for the DFG/CDFG

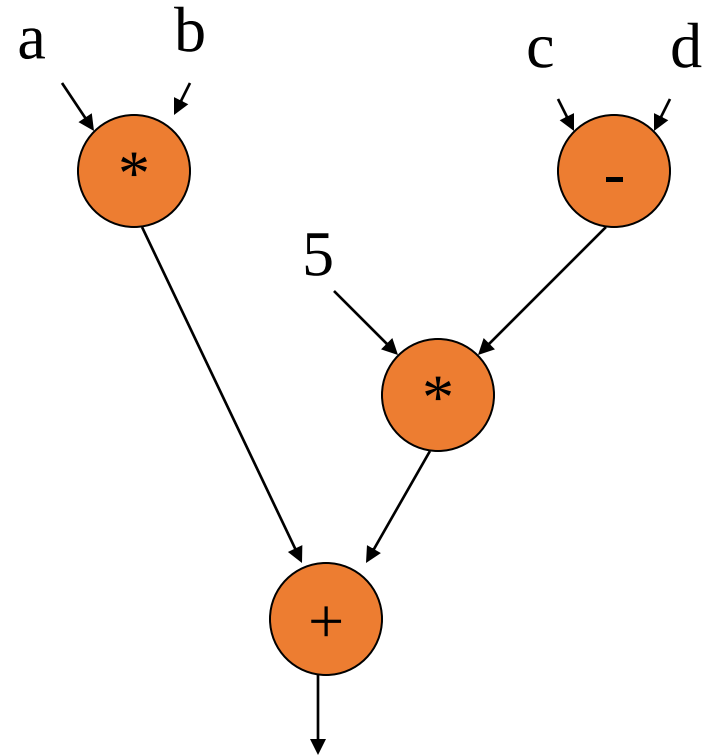
Statement translation and optimization

- Source code is translated into intermediate form such as CDFG.
- CDFG is transformed/optimized.
- CDFG is translated into instructions with optimization decisions.
- Instructions are further optimized.

Arithmetic expressions

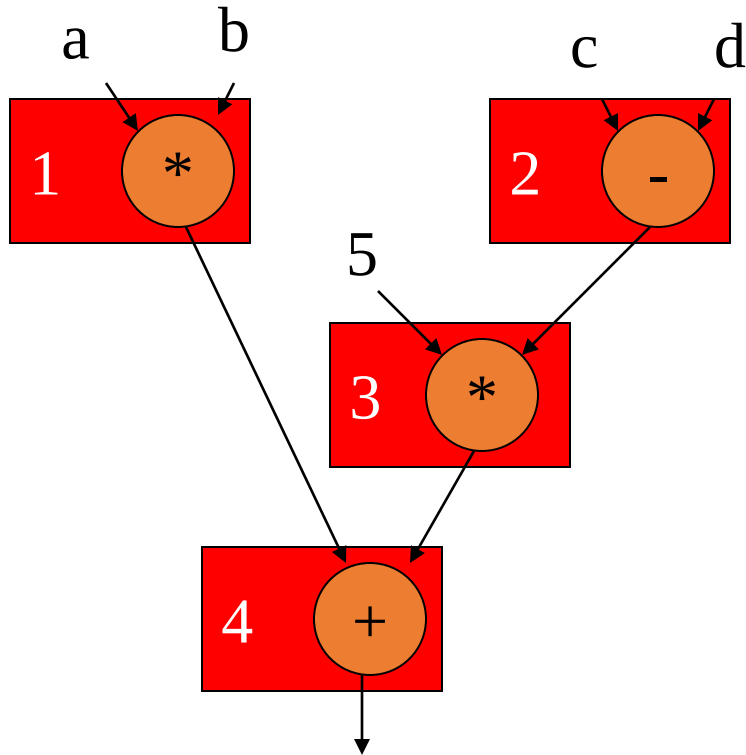
$a*b + 5*(c-d)$

expression



DFG

Arithmetic expressions, cont'd.



DFG

```
ADR r4,a
MOV r1,[r4]
ADR r4,b
MOV r2,[r4]
ADD r3,r1,r2

ADR r4,c
MOV r1,[r4]
ADR r4,d
MOV r5,[r4]
SUB r6,r4,r5
MUL r7,r6,#5
ADD r8,r7,r3
```

code

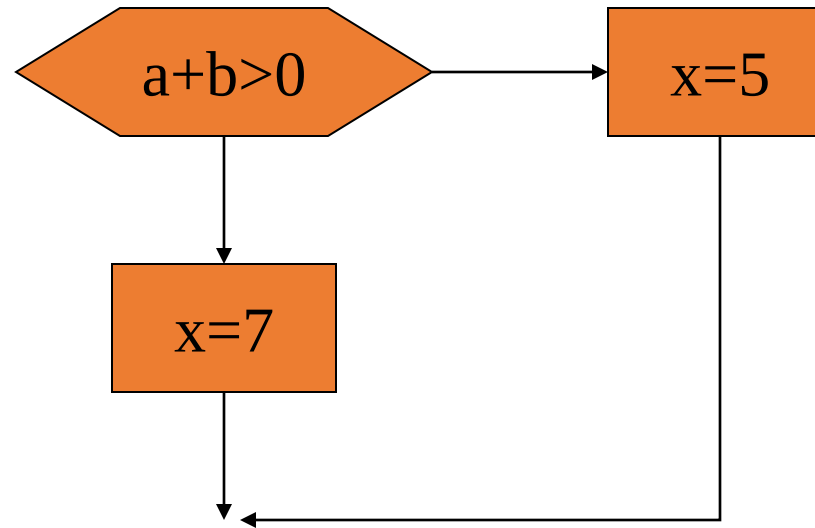
Control code generation

if ($a+b > 0$)

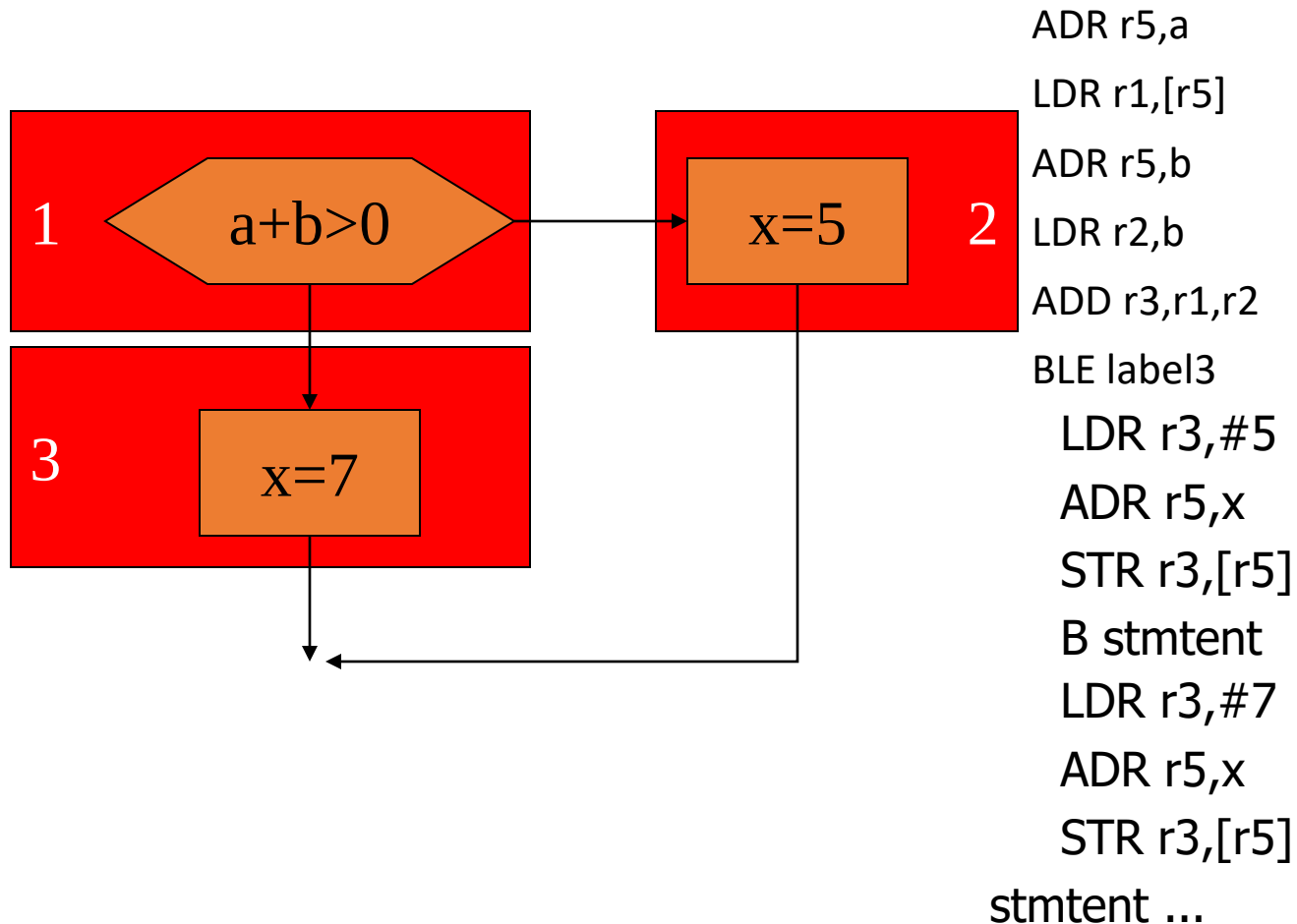
$x = 5$;

else

$x = 7$;



Control code generation, cont'd.

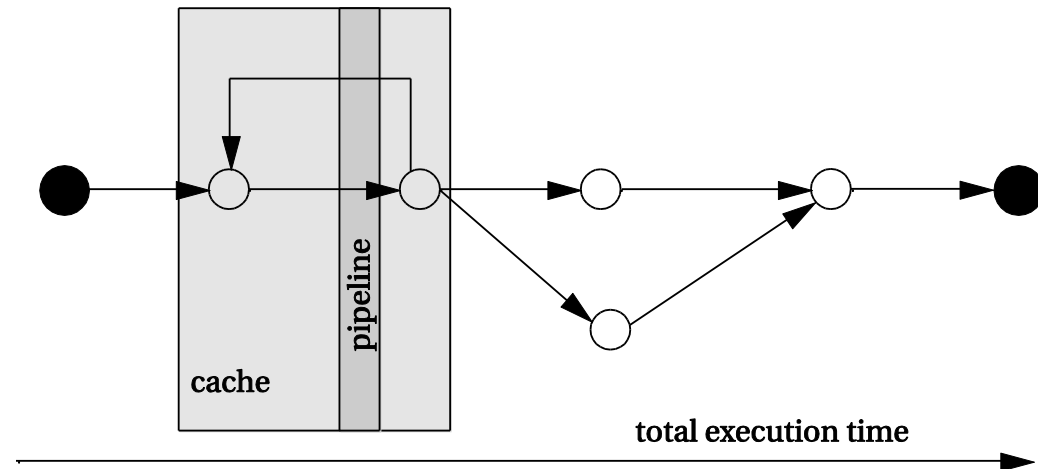


Program design and analysis

- Program-level performance analysis.
- Optimizing for:
 - Execution time.
 - Energy/power.
 - Program size.
- Program validation and testing.

Program-level performance analysis

- Need to understand performance in detail:
 - Real-time behavior, not just typical.
 - On complex platforms.
- Program performance $\neq$ CPU performance:
 - Pipeline, cache are windows into program.
 - We must analyze the entire program.



Complexities of program performance

- Varies with input data:
 - Different-length paths.
- Cache effects.
- Instruction-level performance variations:
 - Pipeline interlocks.
 - Fetch times.

How to measure program performance

- Simulate execution of the CPU.
 - Makes CPU state visible.
- Measure on real CPU using timer.
 - Requires modifying the program to control the timer.
- Measure on real CPU using logic analyzer.
 - Requires events visible on the pins.

Program performance metrics

- Average-case execution time.
 - Typically used in application programming.
- Worst-case execution time.
 - A component in deadline satisfaction.
- Best-case execution time.
 - Task-level interactions can cause best-case program behavior to result in worst-case system behavior.

Elements of program performance

- Basic program execution time formula:
 - execution time = program path + instruction timing
- Solving these problems independently helps simplify analysis.
 - Easier to separate on simpler CPUs.
- Accurate performance analysis requires:
 - Assembly/binary code.
 - Execution platform.

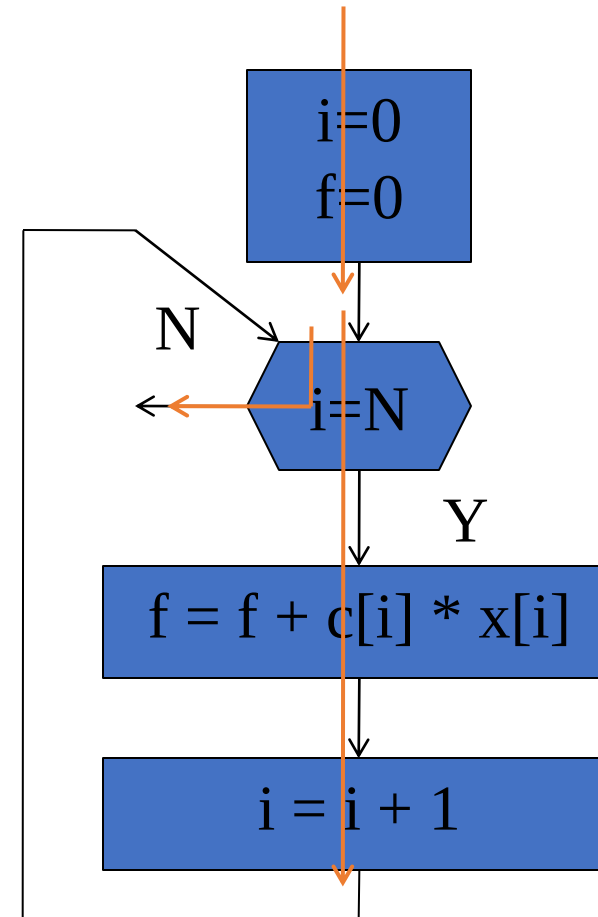
Data-dependent paths in an if statement

```
if (a || b) { /* T1 */  
    if ( c ) /* T2 */  
        x = r*s+t; /* A1 */  
    else y=r+s; /* A2 */  
    z = r+s+u; /* A3 */  
}  
else {  
    if ( c ) /* T3 */  
        y = r-t; /* A4 */  
}
```

a	b	c	path
0	0	0	T1=F, T3=F: no assignments
0	0	1	T1=F, T3=T: A4
0	1	0	T1=T, T2=F: A2, A3
0	1	1	T1=T, T2=T: A1, A3
1	0	0	T1=T, T2=F: A2, A3
1	0	1	T1=T, T2=T: A1, A3
1	1	0	T1=T, T2=F: A2, A3
1	1	1	T1=T, T2=T: A1, A3

Paths in a loop

```
for (i=0, f=0; i<N; i++)  
    f = f + c[i] * x[i];
```



Instruction timing

- Not all instructions take the same amount of time.
 - Multi-cycle instructions.
 - Fetches.
- Execution times of instructions are not independent.
 - Pipeline interlocks.
 - Cache effects.
- Execution times may vary with operand value.
 - Floating-point operations.
 - Some multi-cycle integer operations.

Mesaurement-driven performance analysis

- Not so easy as it sounds:
 - Must actually have access to the CPU.
 - Must know data inputs that give worst/best case performance.
 - Must make state visible.
- Still an important method for performance analysis.

Feeding the program

- Need to know the desired input values.
- May need to write software scaffolding to generate the input values.
- Software scaffolding may also need to examine outputs to generate feedback-driven inputs.

Trace-driven measurement

- Trace-driven:
 - Instrument the program.
 - Save information about the path.
- Requires modifying the program.
- Trace files are large.
- Widely used for cache analysis.

Physical measurement

- In-circuit emulator allows tracing.
 - Affects execution timing.
- Logic analyzer can measure behavior at pins.
 - Address bus can be analyzed to look for events.
 - Code can be modified to make events visible.
- Particularly important for real-world input streams.

CPU simulation

- Some simulators are less accurate.
- Cycle-accurate simulator provides accurate clock-cycle timing.
 - Simulator models CPU internals.
 - Simulator writer must know how CPU works.

Performance optimization motivation

- Embedded systems must often meet deadlines.
 - Faster may not be fast enough.
- Need to be able to analyze execution time.
 - Worst-case, not typical.
- Need techniques for reliably improving execution time.

Programs and performance analysis

- Best results come from analyzing optimized instructions, not high-level language code:
 - non-obvious translations of HLL statements into instructions;
 - code may move;
 - cache effects are hard to predict.

Performance optimization hints

- Use registers efficiently.
- Use page mode memory accesses.
- Analyze cache behavior:
 - instruction conflicts can be handled by rewriting code, rescheduling;
 - conflicting scalar data can easily be moved;
 - conflicting array data can be moved, padded.

Optimizing for program size

- Goal:
 - reduce hardware cost of memory;
 - reduce power consumption of memory units.
- Two opportunities:
 - data;
 - instructions.

Data size minimization

- Reuse constants, variables, data buffers in different parts of code.
 - Requires careful verification of correctness.
- Generate data using instructions.

Reducing code size

- Avoid function inlining.
- Choose CPU with compact instructions.
- Use specialized instructions where possible.